PLSQL – OO

an Object Oriented approach to Table API's

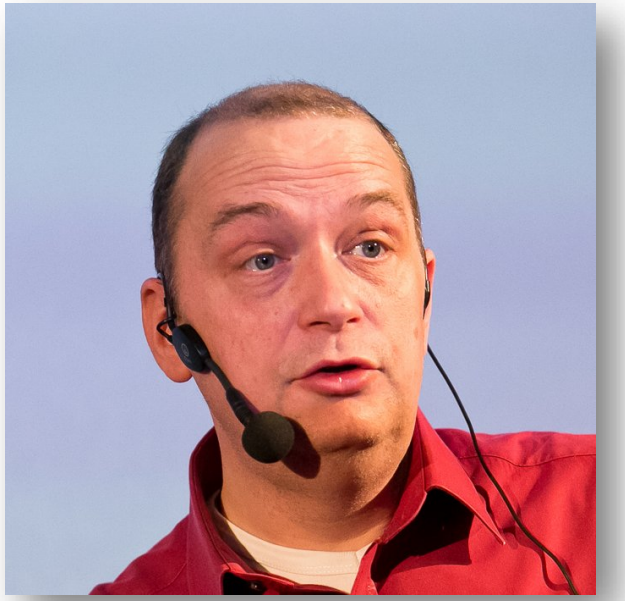
Agenda

- Procedural vs Object Oriented
- PL/SQL a procedural language
- Pipelined functions
- Extending objects
  - Constructor Functions
  - Member Functions & Procedures
- Base Types
- Inheritance
- Sorting
- Exception WHEN_ACCESS_INTO_NULL
- Storing objects in the database
- XML Transitions and OO

Who is Richard Martens

smart4 solutions

- Smart4solutions partner (2017)
- Smart4apex founding member (2010)
- Oracle since 2002 (Oracle 8i)
- PL/SQL, Apex, Web technologies (HTML, CSS, JS, XML, XSLT, REST)
- Special interest in UX
- Trainer at skillbuilders.com
- RIMA on Oracle Forum
- @rhjmartens
- https://richardmartens.blogspot.nl
- https://smart4solutions.nl/blog
- https://bitbucket.org/smart4solutions



ORACLE ACE

SMART4Solutions

Hmm Yes, we’re brothers !!

- Founded 2017
  - Sergei Martens, Robert van Leenders, Richard Martens (all SMART4Apex members)
- Member of the SMART4Unity group
  - SMART4Apex, SMART4Branding
- Nicest APEX family of the Netherlands
  - IT Contracting
  - Projects
  - Training
- Large preliminarily Dutch clients:
  - iTank – VTTI (Worldwide Tank Terminal)
  - Cannock Chase (Credit Management)
  - . . .

Procedural vs Object Oriented

Procedural

- Top – Down
- Focus on “getting the task done”
  - On the structure or procedure of the program
- No inheritance
- (No) Overloading
- Data is shared among the functions of the program

Object Oriented

- Bottom Up
- Focus on data-security
  - Only objects are permitted to access the entities of a class
- Inheritance in three levels (public, private and protected)
- Overloading of functions, constructors and operators (polymorphism)
- Data is shared among objects through the member functions

PL/SQL a procedural language

- Designed as a POP – language (procedural oriented programming)
- Based upon Ada
- OO extensions added in Oracle 8i

```
with Ada.Text_IO;

package body Example is

   i : Number := Number'First;

   procedure Print_and_Increment (j: in out Number) is

      function Next (k: in Number) return Number is
      begin
         return k + 1;
      end Next;

    begin
            Ada.Text_IO.Put_Line ( "The total is: " & Number'Image(j)
);
      j := Next (j);

   end Print_and_Increment;

-- package initialization executed when the package is elaborated
begin

   while i < Number'Last loop

      Print_and_Increment (i);

   end loop;

end Example;
```

Pipelined Functions

AKA's:

- Pipelined functions
- Pipe-row functions
- Function based tables
- Table based functions (strictly this name is incorrect)

1. Specify a type (preferably a DB-type but a packaged type (in spec) suffices
2. Create a table-type

   ```
   Create or replace type <table_type_name> as table of <object_name>;
   ```

3. Create a function that returns the table-type
   1. Each time you want to "print" a row you pipe the object

      ```
      pipe row (t_object_name);
      ```

- DEMO: 01, 02, 03 and 04

Constructor Functions

- Constructor functions return the instance of an object
  - return self as result
- do not “return self;” but instead simply “return;”

- Constructor functions are not inherited
  - because parent type (signature) is different from child type

Extending objects

- We already used a first function, the default constructor function
  - It allowed us to use

    l_emp := new o_emp(empno, ename, job, . . .);

- We can add our own constructor
  - The default constructor remains (overloading)
- We can overrule the default constructor by giving the constructor the exact same "signature"
  - `constructor function o_emp(empno in number, ename in varchar2…) return self as result`

- DEMO: 05, 06

Member Functions & Procedures

- Just like packages we can add our own procedures and functions
  - Saving an object to the database (demo 07)
  - Returning a JSON representation (demo 0080 - 0081)
  - Sending the object to a RESTful API (demo GBILLING)
  - . . .

Static Functions & Procedures

- Static functions and procedures act like packaged functions and procedures
  - You don’t need to instantiate the object first
  - Demo 0085 - 0086

Objects containing other objects

- Until now our attributes were simple datatypes
  - Number, Varchar2, Date etc.
- We can also define an attribute to be of type object
- Demo 09

Base Types and Inheritance

- When defining a type as “NOT FINAL” it can serve as the parent of another type
- That (child-) type is then “UNDER” its parent

- Demo 10

- Be aware, that constructors are never inherited
  - Because the types (the signatures) are not the same

Exception ACCESS_INTO_NULL

- ACCESS_INTO_NULL is raised when the object has no instance
  - It is not “instantiated”
- Demo 11

Method Chaining

- In JavaScript we can chain our commands:
  - ```
    $('#the-element-id')
        .removeClass("xyz")
        .addClass("abc");
    ```
- PL/SQL also allows us to do this:
  - ```
    declare
      l_emp o_emp;
    begin
      l_emp := new(o_emp);
      l_emp.giveRaise(p_percentage => 10).transfer(p_new_dept => 10);
      l_emp.save2db;
    end;
    ```
- Only works for functions ! returning the object type itself
- Demo 12

Storing objects in the database

- Object types can act as the type definition of a column
- The object cannot have a constructor without parameters!!
- Quirky things happen when you select
  - You must select the specific attribute of the object
- Once a type is used as a column source, you should not alter the spec of the type no more. Your data might get corrupted.
  - Altering the body of the type is no problem.
- Demo 13

Sorting

- Two types of functions involved
  - MAP MEMBER FUNCTION return number (Oracle recommended)
  - ORDER MEMBER FUNCTION(p_obj in *objecttype*) return integer
    - Personally I think this is more elegant
- Only ONE function allowed in a type
  - Either one MAP or one ORDER function
- Demo 14, 15

semantic	ORDER function must return
SELF < *argumentObject*	negative number (typically -1)
SELF = *argumentObject*	0
SELF > *argumentObject*	positive number (typically +1)
Undefined comparisson	NULL

XML - Object transitions

- XML transformations to and from Oracle have never been so easy!!
  - XMLTYPE(*<instantiated-object>*) transforms the object to an xmlType
  - *<xmltype>*.toObject(*<object>*) transforms an xml into an instantiated object
  - The object of course must represent the XML perfectly (this is a bit of a nuisance)
- Demo 16, 17

The Pièce de Résistance

- Table API generator for generating objects handling fetching, storing and deleting of records
  - Fetching records

    ```
    l_emp := new o_emp(7839);
    ```

  - Deleting records

    ```
    l_emp := new o_emp(7839);
    l_emp.deletefromdb;
    ```

  - Inserting records

    ```
    l_emp       := new o_emp();
    l_emp.ename := 'Martens';
    l_emp.savetodb;
    dbms_output.put_line('empno: ' || l_emp.empno);
    ```

  - Updating records

    ```
    l_emp      := new o_emp(7839);
    l_emp.comm := 200;
    l_emp.savetodb;
    ```

- Demo 18

Separating Technical from Business Logic

- A table API allows us to separate technical from business logic
- An OO based table API makes it even more readable
- Demo 19